DISK MANAGEMENT

In this chapter we will take a closer look at the design decisions one has to make when managing (hard) disks. We start with a discussion of how a (magnetic) disk works. After that we will look at some disk scheduling algorithms. We end by elaborating on a more recent evolution in disk management, being RAID systems.

1 Magnetic discs

A hard disk consists of one (or more) circular (metal) discs on which information is stored by magnetization. Typically, this information is written and read by a disk head attached to an arm. The disk rotates at a constant speed, usually 3600, 5400, 7200 or 10000 rounds per minute (RPM). This figure is usually correct to within half a percent or less. In practice, the rotation speed will vary slightly around the average. The information on the disk is organized into tracks and sectors. Tracks are concentric circles each consisting of a number of sectors (see Figure 46). Each sector contains a fixed number of bytes, which many disks default to 512 bytes. When data is read or written, it is always in multiples of sectors. Each sector also contains some overhead such as error correction codes, ID information and a synchronization field. When a harddisk consists of multiple disks, we talk about a cylinder when we refer to the set of tracks that are at the same distance from the outside of the disk.

Zone bit recording: In the past, each track contained a fixed number of sectors. This means that on the inner tracks the data is very compact, while on the outer tracks there is a much lower data density. This also implied that the read or write speed was the same all over the disk. The capacity of the discs was determined by the amount of information that could be put on the inner tracks.

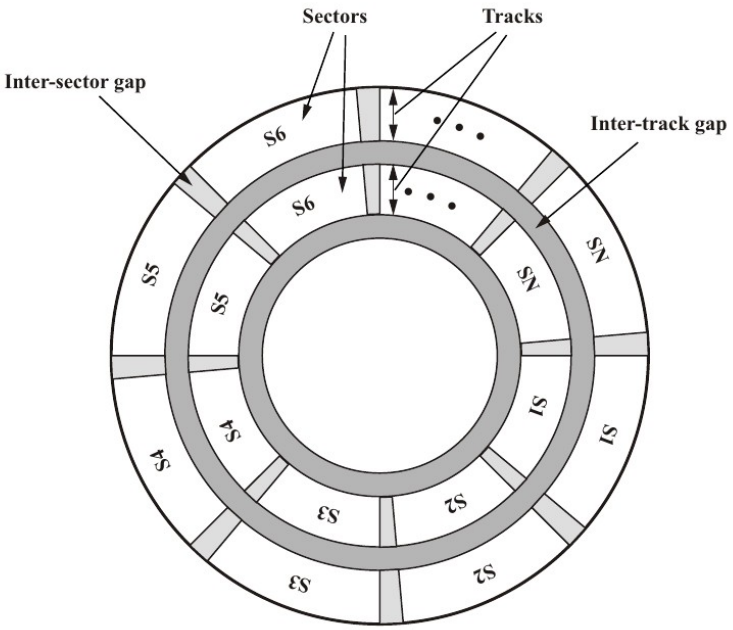

Fig. 46: Disk data layout [Stallings 03].

Today, many discs use zone bit recording, which means that the disc is divided into a number of zones and within each zone, all tracks consist of the same number of sectors. A track on the outside will contain up to twice as many sectors as those on the inside (i.e. several hundred). Because of the constant rotation speed, this means that the writing and reading speed on an outer track is much higher compared to a track on the inner zone. Data is placed on the disk from the outside in. I/O operations on recently written data, when the disk is almost full, can therefore be significantly

slower.

Positioning: When a read or write occurs, the head will first move until it is positioned above the correct track. It then waits until the correct sector (or sectors) present themselves. The time needed for positioning the head is the seek time, while waiting for the sector is called the rotational delay. The time needed for reading or writing itself is the transfer time. Let r be the number of revolutions per second, Ts the seek time, N the number of bytes on the track and b the number of bytes to read/write, then the access time Ta is on average equal to

$$Ta = Ts + 1/2r + b/rN \quad (7)$$

The average rotational delay is equal to 8.33 or 4.16 ms for a 3600 or 7200 rpm disk, respectively. The seek time is usually around 8 to 10 ms (with 15 to 20 ms for a full strike). The speed at which the arm moves, however, is not uniform and depends on the number of tracks over which we move the arm.

A seek is generally made up of four phases: speed-up, coast, slowdown and settle. Thus, it first accelerates (speed-up), then maintains the constant maximum speed (coast), slows down to zero (slowdown) and then the head is perfectly positioned above the track (settle). Very short seeks (maximum two to four cylinders) consist mostly of a settle (about 1 ms), short seeks of a few hundred tracks have no coast phase, while very long seeks consist for a significant part of the coast phase (where time increases linearly with the number of tracks to be run). For the HP C2200A disk (335 MB) with 1449 cylinders, the seek time can be well approximated as follows, where d represents the number of tracks to be run:

$$\text{seek time (ms)} = \begin{cases} 3.45 + 0.597\sqrt{d} & d \leq 616, \\ 10.8 + 0.012d & d > 616. \end{cases}$$

The seek time and rotational delay are often by far the largest contributors to the access time.

When multiple disks are present, the disk heads will all be above the same track (because they are all connected to the same arm). However, only one head will be reading or writing data at any given time. When a read operation requires a change of disk, but not a change of track, there will often be a settling time, because the different disks cannot be made perfectly identical.
The movement of the disk arm is controlled by the disk controller, which must indicate the exact force that the motor driving the arm movement must handle for a given seek. This data is partly stored in a table. To avoid storing all possible values, the disk controller can also use interpolation methods. Due to temperature changes and other factors this table needs to be recalibrated regularly, which takes about a second. This happens automatically every so often (e.g. 1530 minutes) when the machine remains active.

Track skewing and sparing: To keep the disk read speed high when changing tracks, the data of the inner of two adjacent tracks will be skewed by an amount that corresponds to the (worst-case) time needed to change tracks (track skewing). Furthermore, a very small number of sectors will already have errors when the disk is created. These errors are detected by frequent testing in the factory and passed on to the controller in the form of a list. The controller can then map the faulty sectors to an alternative location (sparing).

The disk controller: Like all I/O devices, the hard disk also has a controller that is connected to the bus (allowing data to be exchanged with the processor, main memory, etc). This controller has its own registers and a buffer that can store at least one sector (after all, the reading speed of the disk is usually slower than the speed of the bus). It is the device driver software that places the necessary

requests through the processor into the registers of the controller. If the hard disk is equipped with command queueing, there will be multiple open requests in the hard disk and the controller can handle them independently in a possibly more optimal order. It is therefore important to know exactly how the controller will handle the requests if the operating system software wishes to impose its own order. If the requests are sent to the controller in a sufficiently wide spread, they will of course be processed in order.

Disk caching: Today, a disk can also have its own cache. This cache will mainly follow a read-ahead strategy when reading, also reading the sectors following the requested sector(s) into the cache to immediately handle possible future requests. Unlike other caches, data read from this cache will often be deleted immediately (file reading is therefore much more sequential in nature than code walking). When the read-ahead operations exceed a track boundary, one speaks of aggressive read-ahead. The latter can be disadvantageous because a track change takes time and cannot be interrupted.

Old harddisks sometimes used on-arrival read-ahead. The track was read completely and the read operation started as soon as the disk head was positioned above the track. For operations that required a large part of the track to be read, this could save a lot of time, because there was no waiting for the start of the area to be read. However, with more and more sectors on a single track and the average size of a read operation barely increasing, it's not really worth supporting on-arrival read-ahead anymore.
For writing, caching is also very useful because the same data can be overwritten in the cache without having to perform a write operation on the disk in between. Furthermore, the controller itself can determine the order in which the data is written to the disk. Some caution is necessary though, as the cache memory may be volatile, indicating that the contents are lost when the cache is not powered.

2 Disk Scheduling

The above considerations regarding the average access time Ta apply mainly to random requests, i.e., the successive locations we visit on the disk are at random positions. We can reduce the time lost due to arm movement and rotational delay by handling the reads and writes in a certain order.

First-In-First-Out (FIFO): The most obvious and fair order is FIFO. FIFO can give rise to good performance if many requests are clustered on a limited number of neighboring tracks. However, in many cases FIFO will give rise to a low performance that is not much better than random scheduling. In Figure 47(a), we see an example where 8 requests are handled in FIFO order. In total, the arm has to complete 640 tracks, indicating that the FIFO order is indeed not very efficient.

Shortest Seek Time First (SSTF): This algorithm will take the current position of the disk head into account. The next request that is processed, is the one that requires the shortest seek time. Since the speed of the head to switch tracks is not linear in function of the number of tracks, an estimate is required to determine the seek time. Although this technique can realize a high utilization (and exploits locality of reference properties), it can lead to starvation. The disk may hang around the same sectors for a very long time, causing other requests to wait unacceptably long. This problem mainly occurs in systems where the disk is heavily loaded with reads and writes.

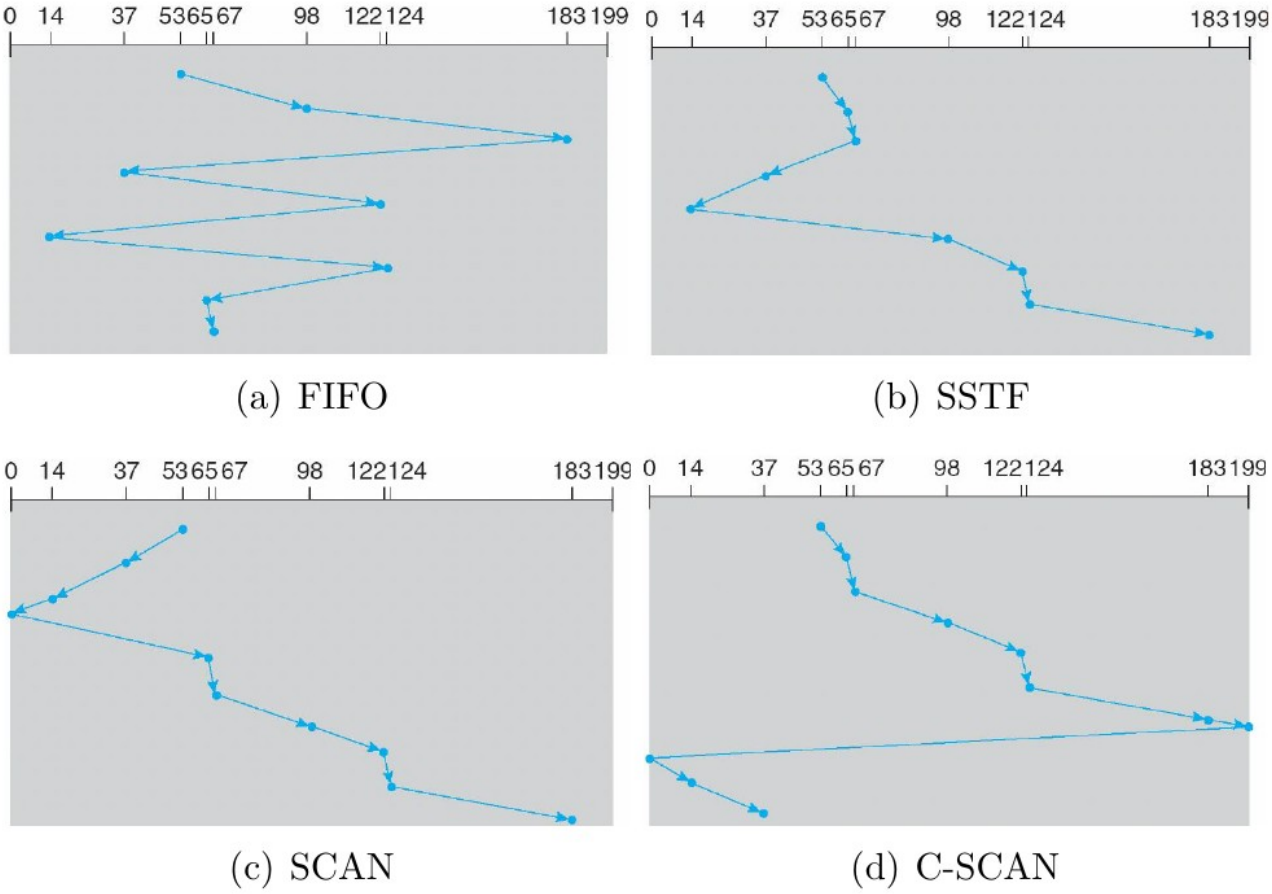

Fig. 47: Disk scheduling with queue = 98, 183, 37, 122, 14, 124, 65, 67 and with the arm at position 53 at the start

In Figure 47(b) we see an example of SSTF scheduling, where for simplicity we assume that the arm moves at a constant speed. In total, only 236 tracks are completed, in contrast to the 640 tracks of the FIFO sequence. Although the SSTF order is very similar to the shortest process next (SPN) scheduling order, it is not optimal because handling a first request affects the time required to handle the remaining requests. The LOOK algorithm, which we discuss further, will handle the 8 requests in 208 track switches.

SCAN and Cyclical SCAN (C-SCAN): The SCAN policy addresses the starvation effect of SSTF by limiting the possible arm movement. That is, the arm may only change direction upon reaching the outside or inside of the disk. Studies with random workloads have shown that SCAN achieves much lower response time variances, but requires a slightly higher average time. The SCAN technique has the disadvantage that the tracks in the middle of the disk are visited more regularly than those on the outside or inside. This implies that the performance characteristics differ based on the position of the data, which is not desirable. To nullify the inequality created by SCAN, one can also require that when the arm reaches the inside of the disk, it must immediately return completely to the outside before handling new requests. This is the way the C-SCAN algorithm works.

LOOK and Cyclical LOOK (C-LOOK): The LOOK algorithm is a variant of SCAN, where the disk head will change direction when there are no requests outstanding in the direction the head is currently moving. Although a bit more complex, this can lead to a small performance improvement. C-LOOK is, as you might expect, the variant of LOOK which, like C-SCAN, gets rid of the inequality due to the track location.

VSCAN(R): This algorithm, which has $0 \leq R \leq 1$ as input parameter, will choose the request closest to the current position. However, the metric for measuring the distance is no longer the physical distance (as with SSTF). Depending on whether the request required a change in the

direction the arm was moving or not, the physical distance is added R times the full width of the disk, so requests that require a change in direction are considered further away. Thus VSCAN(0) is equivalent to SSTF, VSCAN(1) to LOOK (and not SCAN). Experiments have shown that VSCAN(0.2) gives a good balance between low average seek times and the degree to which starvation occurs.

FSCAN: Another effect that can occur with the SCAN method is arm stickiness. When many requests come in for the same track, the arm can hang there for a long time. As long as the arm is positioned above this track, all new requests for this track will also get priority over the other outstanding requests. One can partially solve this by using a two stage buffer. All requests are collected in a first buffer. When a new set of requests is scheduled, all requests are pushed to the first buffer. Next, the SCAN procedure is applied to this set. The next set of requests to be scheduled, is meanwhile temporarily held back and stored in the second buffer.

Finally, it should be noted that modern computer systems also have a disk cache. This will make successive read operations much faster. The locality of reference principle is therefore mainly exploited by the cache and to a lesser extent by the scheduling strategy. Furthermore, the file allocation method can also strongly influence the performance of the disk.

3 RAID: redundant arrays of independent discs

An important development in disk management, which has become a multi-billion dollar business, is the introduction of Redundant Arrays of Independent Discs (RAID) by Patterson, Gibson and Katz in their 1988 SIGMOD paper. The underlying idea is to set up a high performance and high capacity disk system using many inexpensive disks. However, it is important that the reliability of such a system is equal to that of a classical single hard disk. Finally, in the case of N disks, the average time to failure will be reduced by a factor of N.

RAID systems, as in the original paper, are divided into several levels, ranging from RAID 0 to RAID 7. There are also multilevel systems. We start with the simplest RAID level, which is actually an AID (and not a RAID) as it does not contain redundancy.

When we talk about the performance of a RAID system, we mainly look at the read and write speed, making a distinction between random reads and sequential reads. Random reads consist of many small read operations on different locations on the (logical) disk. Sequential reads are long read operations in which the seek and rotation time play a less important role. The same division can be made for writes. Robustness is also taken into account. Can the system handle crashes, what is the remaining performance in this case and how easy can the system be restored?

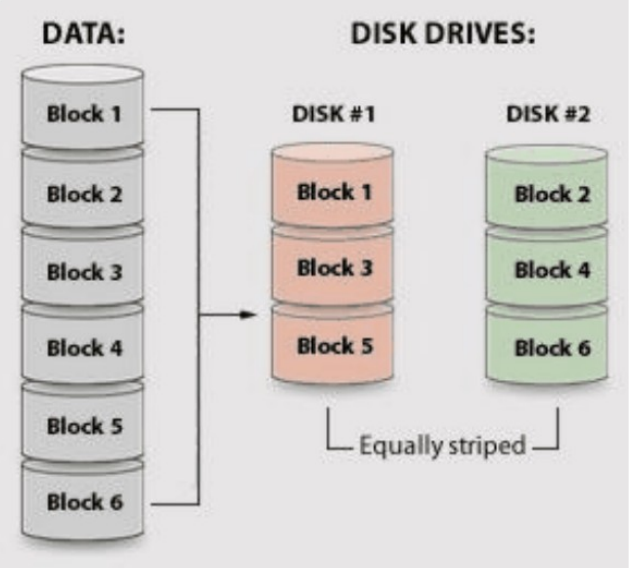

Fig. 48: RAID level 0

3.1 RAID 0

When using RAID level 0, we divide all data into a number of blocks. These blocks are then spread across N disks, by writing blocks i, N + i, 2N + i, etc. on disk i (see Figure 48). Blocks 1 to N are called a stripe, as are blocks N + 1 to 2N , 2N + 1 to 3N , and so on. This is why people often refer to RAID 0 with the term stripe.

An important parameter of striping is the stripe length (which is equal to N times the block size). Large stripes are often useful to support random reads, because then, when the controller supports it, multiple (short) read operations can take place simultaneously (one on each disk). Shorter stripes, where data from a single read is spread across multiple disks, are useful for sequential read operations, as a single read can occur simultaneously. Note that the seek and rotation time is slightly larger in this case, because we need to calculate the time of the most removed head. Also writes are done very efficiently in a RAID 0 system, where the same remarks apply to the stripe length.

The robustness is of course very poor. If one disk fails, the whole system is down (even more so with short stripe lengths). Therefore, this solution is only justified for systems that require very high performance (without being too expensive) and for which the data is not very important or is regularly backed up. Furthermore, it is best that both (all) disks have the same properties, e.g. storage capacity.

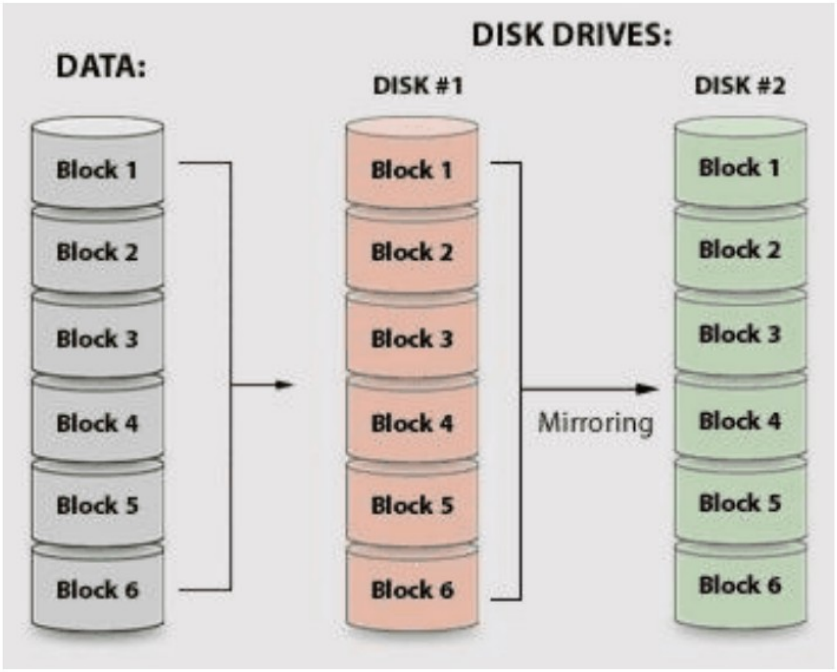

Fig. 49: RAID level 1

3.2 RAID 1

RAID level 1 is very different from level 0 and is especially robust, and to a lesser extent, performs very well. The main disadvantage of this solution is its price. With RAID 1 all data is placed on each disk, which is why this is called mirroring (see Figure 49). In practice, this configuration almost never involves more than two disks.

It is immediately clear that this system has a good reliability. If a disk fails, the system remains operational and works as a single disk system. Restoring is also quite easy, you only need to make a new copy.

Random read operations in RAID 1 are slightly faster, because the load can be spread over both disks. For sequential reads, there is usually little gain over a single disk, as a single large read will happen on one disk. Writes must be done on both disks, making the write operation slightly slower than single disk systems. Namely, the largest seek and rotation time of both disks determines the execution time. With sequential writes, this is less important. So when a disk fails, the write performance will slightly increase, in contrast to the reads.

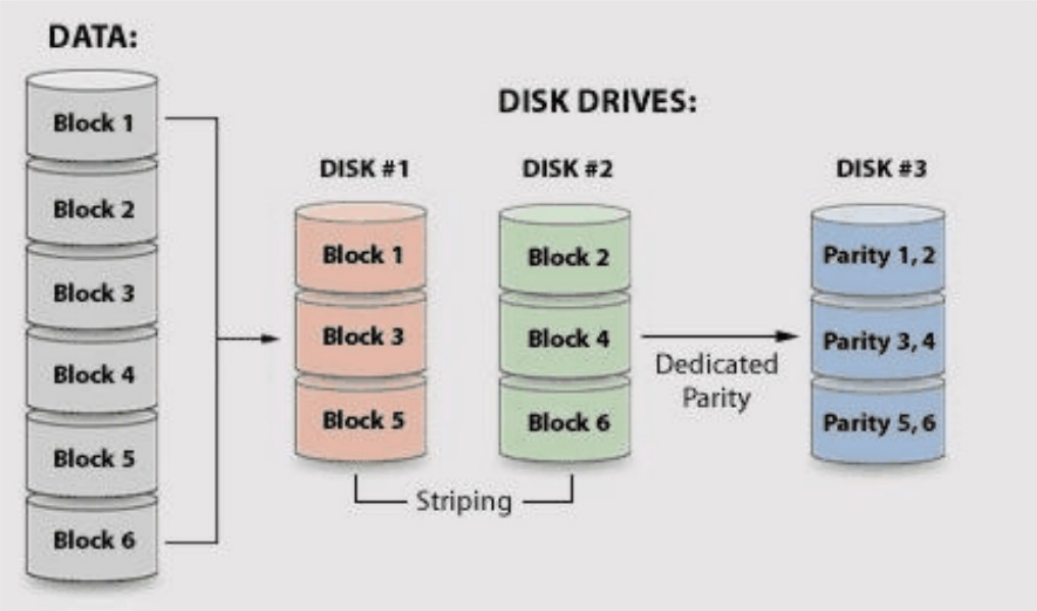

Fig. 50: RAID level 3

The costs for RAID 1 systems are very high, so this is mainly used when the data is very important (e.g. financial data) and performance is less important.

3.3 RAID 3 and 4

RAID 3 and 4 work analogously to RAID 0, meaning that data is striped across two (or more) disks. However, an additional disk containing parity data is also provided. Parity data is one of the simplest forms of error correction coding. To determine the value of the parity bits, we add up all blocks that are part of the same stripe (where 1 + 1 = 0). So the i-th bit of the parity block is 0 (1) if the sum of the i-th bits of each of the blocks is even (odd).

In the case of RAID 3, the stripes are very short (less than 1024 bytes in a stripe), so each read (or write) operation will use all the disks simultaneously. The heads of the disks are also synchronized in RAID 3 (so they are all on the same track and sector). Sequential read operations are very fast. Also writes are quite efficient because we usually write a whole stripe at once, which allows us to calculate the parity data based on the input. Random reads and writes are slightly less efficient due to the very short stripes.

RAID 4 is analogous to RAID 3, but uses large stripes and will not synchronize the heads of the disks. This improves the performance of random read operations. However, a random write operation will often only change one block of a stripe. This means that we first have to compare the old and new data and where there are differences, add these to the old parity data to obtain the new parity data. A single write thus physically requires 2 write and 2 read operations, which is very

expensive for systems with many writes. Sequential reads and writes are slightly less smooth than with RAID 3.

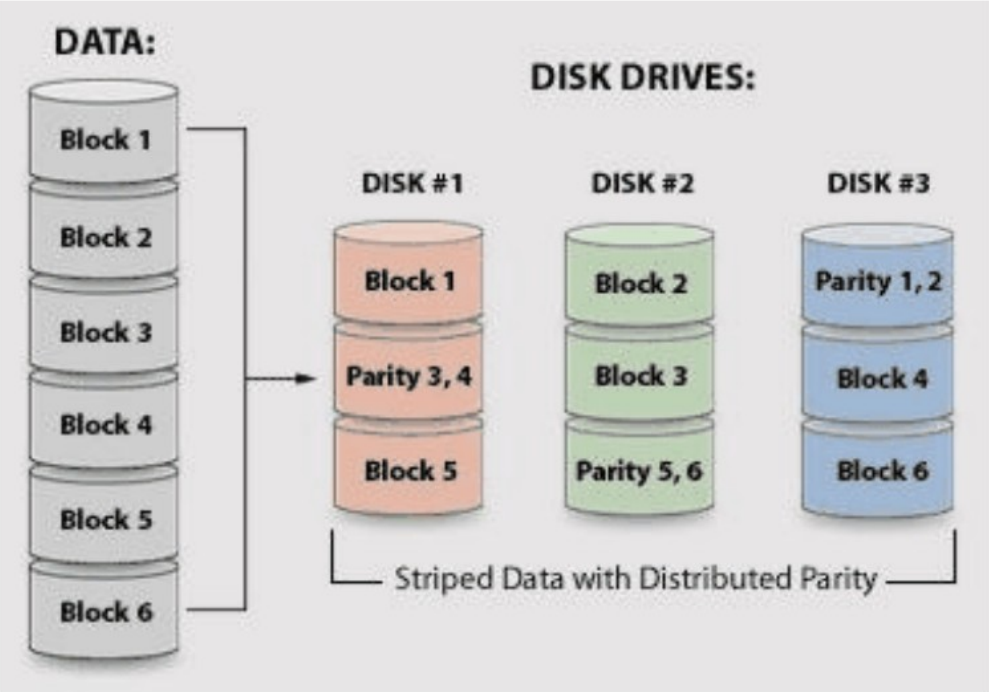

Fig. 51: RAID level 5

The robustness of RAID 3 and 4 systems is similar. The system remains operational when a disk fails, because the information on the disk that failed can be reconstructed from the remaining data (by taking the sum of the remaining blocks). However, when reading data, the missing data often needs to be determined. This means that requesting a single block can result in many read operations (one on each remaining disk). In RAID 3 this is less dramatic, as the small stripes are usually read completely anyway (and the heads are synchronized). In RAID 4, the data may come from a different disk than the one that failed. However, if the requested information is on this disk, it will require a lot of extra work. So the consequences of a RAID 3 failure are not as bad as a RAID 4 failure.
Both systems have the major disadvantage that all parity data is located on the same disk, which means that it is involved in every write operation.

If many writes occur, this disk is a potential bottleneck. RAID 5 solves this problem. Finally, we note that RAID level 2 differs from level 3 by using Hamming codes (instead of parity checks) to protect the data. Although this solution is more robust, it requires more extra disks and the calculation of the control data is more work-consuming. For these reasons, there are virtually no commercial RAID 2 solutions.

3.4 RAID 5 and 6

RAID level 5 removes the bottleneck present by the parity disk in RAID 4 systems by spreading the parity data across all the disks (see Figure 51). This will slightly improve the performance of random writes. However, in case a disk fails, there is still a clear performance degradation.

RAID 5 is probably the most popular RAID level we find in computer systems today. This also makes it more financially attractive. By spreading the parity data, random reads can even be more efficient than with a level 0 configuration (because there is an extra disk). The extra cost of a write remains, but there is now also robustness compared to level 0. Software solutions of RAID 5

systems can seriously threaten the performance because of the high amount of parity calculations.

RAID level 6 is an extension of level 5, where an extra disk is added. In addition to the parity check, an extra block of control information is created per stripe. This allows the system to survive a double disk failure.

3.5 RAID X + Y

Not long after the introduction of RAID levels 0 to 6, configurations were set up that combined multiple levels, in order to exploit the benefits of both. This has given rise to a number of successful combinations, which are of course somewhat more complex in operation and implementation. When we combine level X and Y (as far as this is possible), we first divide the data into sets using RAID level Y. In each set we then apply RAID level X and we note this as RAID X + Y . For example, if we apply RAID 0 + 1, we first use mirroring and then strip in each mirror (see Figure 52). The alternative sequence, i.e. RAID 1 + 0 with striping first and then mirroring, gives rise to the same physical data distribution. However, the operation of both schemes is different (see below).

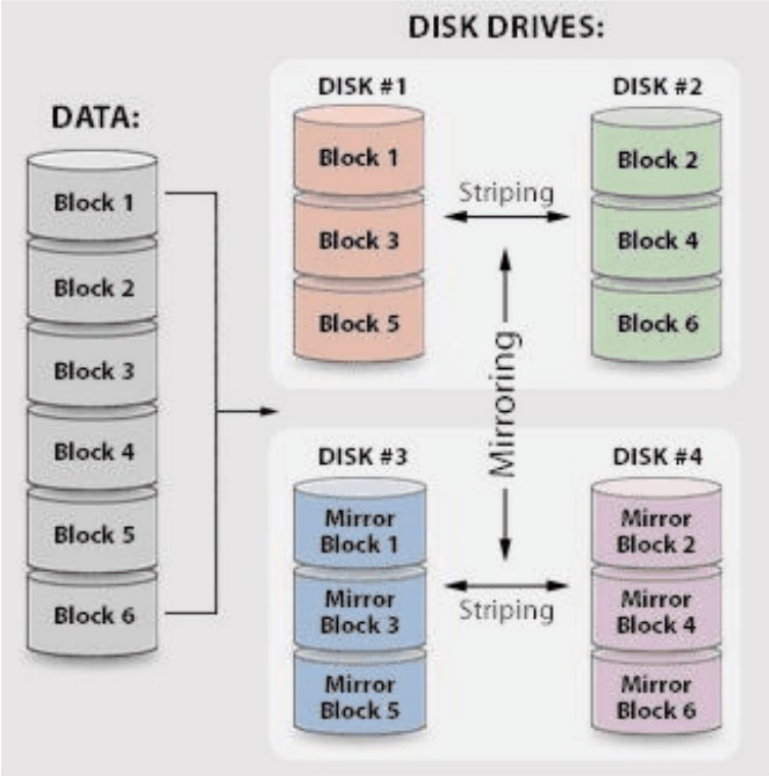

Fig. 52: RAID level 0+1

The naming of multiple RAID levels leaves much to be desired in the literature. For example, RAID X + Y is also listed as XY and X/Y, but also as Y X, Y /X or even Y + X. It is therefore very important to check at the time of purchase which combination is actually used. The order differs even among products of the same manufacturer. In addition, the 0 + 3 scheme is sometimes referred to as 53, which leads to confusion.

RAID 0 + 1 and 1 + 0

When we combine RAID 0 and 1, we usually do this to link the performance of RAID 0 to the reliability of RAID 1. With RAID 0 + 1, we first take a mirror of all the data and then divide each mirror separately over 5 disks by applying striping. Whenever a read or write operation takes place, the controller selects a mirror and uses striping to read or write data from this mirror.

When we apply RAID 1 + 0, reads and writes are slightly different. The data is again spread over 5 disks and duplicated, but now the controller will first use stripping and secondly select a mirror for all affected blocks from the stripe.

In terms of performance, there is little difference between the two sequences. Read operations are very efficient, write operations a little less because each write has to happen in two places. The big difference however is in the robustness. If we have 2N disks and 1 disk fails in RAID 0 + 1, the controller will handle all requests via the mirror in which no error occurred. Thus, performance decreases significantly because all other disks that are also part of the mirror in which the failure occurred are no longer used. If another disk in the remaining mirror fails, the system is rendered unusable (but often repairable).

RAID 1 + 0 is more robust because if one disk fails, all of the remaining 2N - 1 disks will still be in use (because the mirror is chosen for each block in the second instance). In fact, as long as no two disks containing the same data fail, the system can remain operational. Thus, RAID 1 + 0 is much more robust, as the RAID 0 + 1 controller is not intelligent enough to use the remaining disks in the failed mirror.

RAID 0 + 5 and 5 + 0

If we have $N * M$ disks, with $N \geq 2$ and $M \geq 3$, then we can create a RAID 0 + 5 or 5 + 0 setup. With RAID 0 + 5, we have M RAID 0 sets of N disks each, which together form a RAID level 5. Or, we first create M logical disks to which we apply RAID 5 and then divide the contents of each logical disk among N RAID 0 disks. For RAID 5 + 0, on the other hand, we form N RAID 5 sets with M disks each.

Write operations are somewhat slower than with RAID 0 + 1 or 1 + 0 due to parity, but the disks are used more efficiently. Namely, with RAID 0 + 1 or 1 + 0, only half of the storage capacity is used, while RAID 0 + 5 and 5 + 0 uses (M - 1)/M . However, the number of disks can often be written in multiple ways as $N * M$. Suppose we have $21 = 3 * 7 = 7 * 3$ disks available. Then with RAID 5 + 0 we can either form 7 RAID 5 sets of 3 disks each, or 3 RAID 5 sets of 7 disks each. In the first case, 67% of the capacity is used, in the second case, 86%. The choice with more overhead is of course more robust.

There are many other useful combinations such as 0 + 3, 3 + 0, 1 + 5 or 5 + 1, each characterized by its own advantages and disadvantages.